

We first need to determine which, if any, triangles in the existing DT are now invalid because their circumcircle contains the `point_to_add`. We'll do the following:

1. Use a loop to iterate over every triangle in the existing DT.
2. For each triangle, find the circumcenter and radius of its circumcircle.
3. Find the distance between the `point_to_add` and this circumcenter.
4. If this distance is less than the circumradius, then the new point is inside the triangle's circumcircle. We can then conclude this triangle is invalid and needs to be removed from the DT.

We can accomplish these steps with the following code snippet:

```
import math
invalid_triangles = []
delaunay_index = 0
while delaunay_index < len(delaunay):
    circumcenter, radius = triangle_to_circumcenter(delaunay[delaunay_index])
    new_distance = get_distance(circumcenter, point_to_add)
    if(new_distance < radius):
        invalid_triangles.append(delaunay[delaunay_index])
    delaunay_index += 1
```

This snippet creates an empty list called `invalid_triangles`, loops through every triangle in our existing DT, and checks whether a particular triangle is invalid. It does this by checking whether the distance between the `point_to_add` and the circumcenter is less than the circumcircle's radius. If a triangle is invalid, we append it to the `invalid_triangles` list.

Now we have a list of invalid triangles. Since they are invalid, we want to remove them. Eventually, we'll also need to add new triangles to our DT. To do that, it will help to have a list of every point that is in one of the invalid triangles, as those points will be in our new, valid triangles.

Our next code snippet removes all invalid triangles from our DT, and we also get a collection of the points that make them up.

```
points_in_invalid = []

for i in range(len(invalid_triangles)):
```